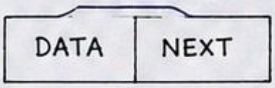
1. Singly Linked List Operations

A Singly Linked List is a linear data structure in which each node contains two parts : DATA and ADDRESS (of next node).

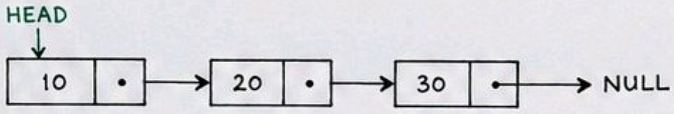
Node Structure :



a) Insert element at the beginning

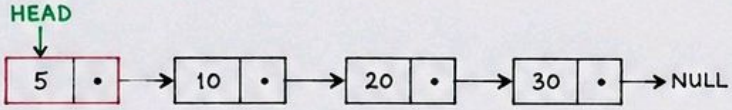
- Create a new node.
- Insert data in the node.
- Make new node point to the current first node.
- Move HEAD to the new node.

Before Insertion :



Insert 5 at beginning

After Insertion :



Algorithm :

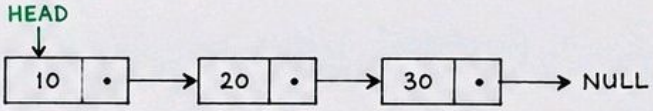
1. Create NEW node
2. NEW → data = value
3. NEW → next = HEAD
4. HEAD = NEW

Time Complexity : O(1)

b) Insert element at the end

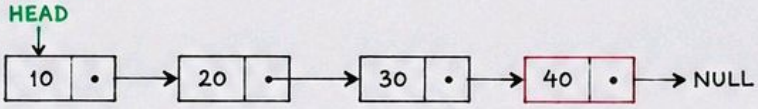
- Create a new node.
- Insert data in the node.
- Set new node's next as NULL.
- Traverse till last node.
- Connect last node to new node.

Before Insertion :



Insert 40 at end

After Insertion :



Algorithm :

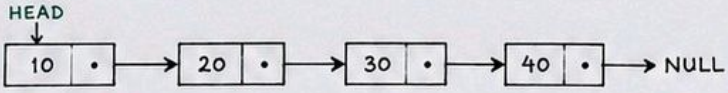
1. Create NEW node
2. NEW → data = value
3. NEW → next = NULL
4. Traverse till last node
5. last → next = NEW

Time Complexity : O(n)

c) Delete the specified element

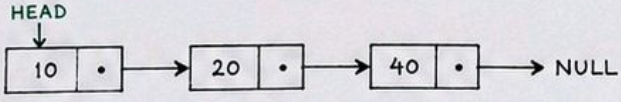
- Search the node to be deleted.
- Store the address of previous node.
- Change the link of previous node.
- Delete the required node.

Before Deletion :



Delete 30

After Deletion :



Algorithm :

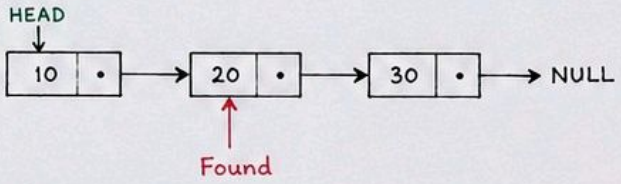
1. Find the node to be deleted
2. prev → next = current → next
3. Delete current node

Time Complexity : O(n)

d) Search the given element

- Start from HEAD.
- Compare each node data with the given element.
- If match found, return position (or FOUND).
- If not found till the end, return NOT FOUND.

Search for 20



Algorithm :

1. Start from HEAD
2. Compare node's data with key
3. If match → FOUND
4. Else move to next node
5. Repeat until NULL

Time Complexity : O(n)

2. Doubly Linked List Operations

A Doubly Linked List is a linear data structure in which each node contains three parts : previous pointer (prev), data part and next pointer (next).



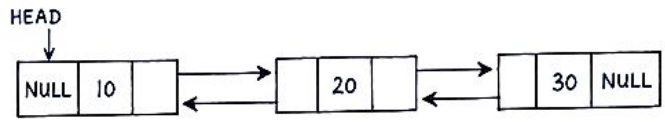
Structure of a node

i) Add an element at the beginning of the list.

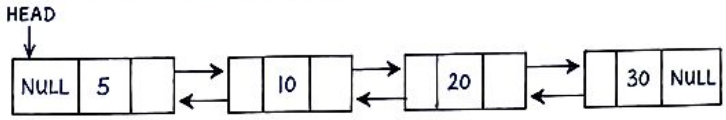
Algorithm :

1. Create a new node.
2. Insert the data into the node.
3. Set new -> prev = NULL.
4. Set new -> next = head.
5. If list is not empty, set head -> prev = new.
6. Move head = new.
7. Stop.

Before Insertion



After Insertion (insert 5)



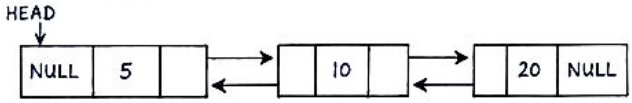
---

ii) Delete an element at the beginning of the list.

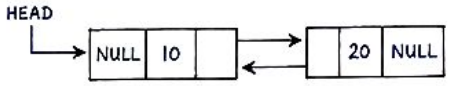
Algorithm :

1. Check whether the list is empty.
2. Store first node in temporary pointer temp.
3. Move head = head -> next.
4. Set head -> prev = NULL.
5. Delete temp.
6. Stop.

Before Deletion



After Deletion



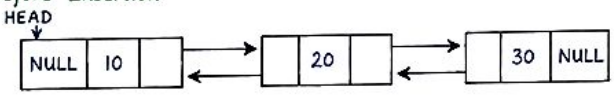
---

iii) Add an element after an item of the list.

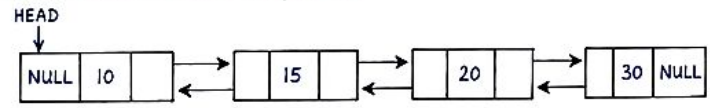
Algorithm :

1. Search the required item in the list.
2. Create a new node.
3. Insert data into new node.
4. Set new -> next = temp -> next.
5. Set new -> prev = temp.
6. Set temp -> next = new.
7. If next node exists, set its prev = new.
8. Stop.

Before Insertion



After Insertion (insert 15 after 10)



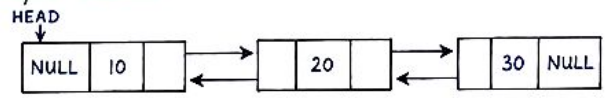
---

iv) Delete an element from the end of the list.

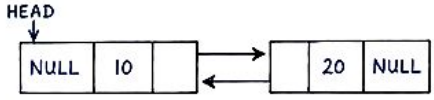
Algorithm :

1. Check whether the list is empty.
2. Traverse to the last node.
3. Store last node in temp.
4. Move to previous node.
5. Set previous node next pointer as NULL.
6. Delete temp.
7. Stop.

Before Deletion



After Deletion



Advantages :

1. Traversal can be done in both forward and backward direction.
2. Insertion and deletion operations are easy.

Disadvantage :

1. Requires extra memory for previous pointer.

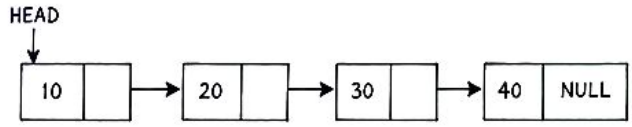
a) How a Single Linked List can be Reversed

Reversing a singly linked list means changing the links between nodes so that the last node becomes the first node.

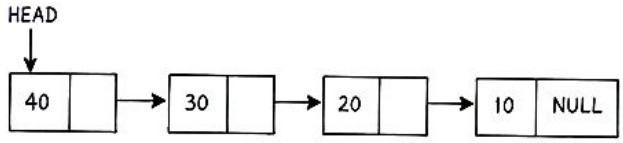
Algorithm :

1. Set three pointers : prev = NULL, curr = head, next = NULL.
2. Traverse the list.
3. In each step, store next node in next.
4. Change curr -> next to prev.
5. Move prev to curr.
6. Move curr to next.
7. Repeat steps 3 to 6 until curr becomes NULL.
8. Update head = prev.
9. Stop.

Before Reversing



After Reversing



Dry Run :

Step	prev	curr	next	Operation
Initial	NULL	10	NULL	Initialize pointers
1	NULL	10	20	next = curr->next
2	NULL	10	20	curr->next = prev
3	10	20	–	prev = curr
4	10	20	30	curr = next
5	10	20	30	next = curr->next
6	10	20	30	curr->next = prev
7	20	30	–	prev = curr
8	20	30	40	curr = next
9	20	30	40	next = curr->next
10	20	30	40	curr->next = prev
11	30	40	–	prev = curr
12	30	40	NULL	curr = next (NULL)
13	30	NULL	–	Exit loop
14	40	NULL	–	head = prev

Now, head points to the new first node (40).

Explanation :

- We use three pointers to reverse the links.
- In each iteration, we store next node, reverse the link, and move forward.
- When curr becomes NULL, prev will point to the new head of the reversed list.
- Finally, update head = prev.

Time Complexity : O(n)

Space Complexity : O(1)

---

b) List and explain the Applications of Linked List

A linked list is a linear data structure in which elements (nodes) are not stored at contiguous memory locations. Each node contains data and address of the next node.

Applications :

1. **Dynamic Memory Allocation :**
   Linked lists are used to implement dynamic memory allocation in operating systems.
2. **Implementation of Stacks and Queues :**
   Stacks and queues can be efficiently implemented using linked lists.
3. **Polynomial Representation :**
   Polynomials can be represented as linked lists where each node stores coefficient and exponent.
4. **Sparse Matrix Representation :**
   Linked lists are used to store non-zero elements of a sparse matrix to save memory.
5. **Graph Representation :**
   Adjacency lists in graphs are implemented using linked lists.
6. **Undo Functionality :**
   Linked lists are used in applications like text editors and browsers to implement undo operations.
7. **File System Implementation :**
   Linked lists are used to manage files in file systems.

4. CIRCULAR LINKED LIST : STANDARD OPERATIONS

In a circular linked list, the last node points back to the first node, forming a circle.

Node Structure

data	next

1. CREATE (Empty List)

Pseudo code :

```
1. START
2. SET rear = NULL
3. STOP
```

Explanation :

If rear = NULL, it indicates that the list is empty.

2. CHECK EMPTY

Pseudo code :

```
1. START
2. IF rear = NULL
       RETURN TRUE
3. ELSE
       RETURN FALSE
4. STOP
```

Explanation :

If rear is NULL, the list has no nodes.

3. TRAVERSAL (Display)

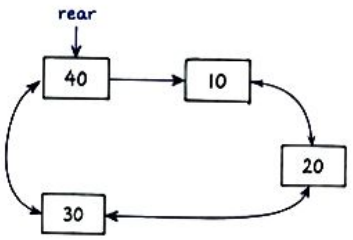
Pseudo code :

```
1. START
2. IF rear = NULL
       PRINT "List is empty"
       STOP
3. SET temp = rear -> next   // first node
4. REPEAT
       PRINT temp -> data
       SET temp = temp -> next
   UNTIL temp = rear -> next
5. STOP
```

Explanation :

Start from the first node and move forward until we reach the first node again.

Diagram :



4. INSERT AT BEGINNING

Pseudo code :

```
1. START
2. CREATE new node
3. READ new -> data
4. IF rear = NULL   // list empty
       new -> next = new
       rear = new
   ELSE
       new -> next = rear -> next
       rear -> next = new
5. STOP
```

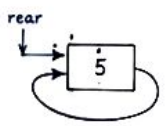
Explanation :

If list is empty, the new node points to itself and becomes rear.

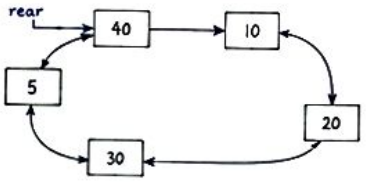
Otherwise, insert new node before the first node.

Diagrams :

(a) Inserting 5 in an empty list



(b) Inserting 5 in non-empty list



5. INSERT AT END

Pseudo code :

```
1. START
2. CREATE new node
3. READ new -> data
4. IF rear = NULL   // empty list
       new -> next = new
       rear = new
   ELSE
       new -> next = rear -> next
       rear -> next = new
       rear = new
5. STOP
```

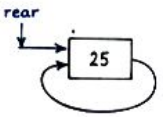
Explanation :

If list is empty, new node becomes the only node.

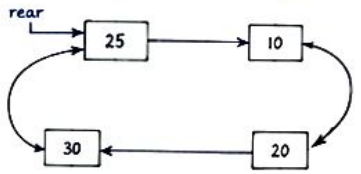
Otherwise, insert new node after rear and update rear.

Diagrams :

(a) Inserting 25 in empty list



(b) Inserting 25 in non-empty list



6. DELETE FIRST NODE

Pseudo code :

```
1. START
2. IF rear = NULL
       PRINT "List is empty"
       STOP
3. SET temp = rear -> next   // first node
4. IF rear -> next = rear    // only one node
       FREE(temp)
       rear = NULL
   ELSE
       rear -> next = temp -> next
       FREE(temp)
5. STOP
```

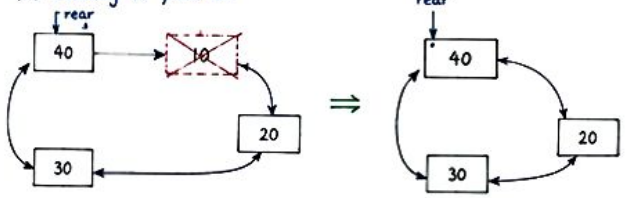
Explanation :

If only one node, make list empty.

Otherwise, remove the first node and link rear to the new first node.

Diagrams :

(a) Deleting 10 from list



7. DELETE LAST NODE

Pseudo code :

```
1. START
2. IF rear = NULL
       PRINT "List is empty"
       STOP
3. SET temp = rear -> next   // first node
4. IF rear -> next = rear    // only one node
       FREE(rear)
       rear = NULL
   ELSE
       WHILE temp -> next != rear
           temp = temp -> next
       temp -> next = rear -> next
       FREE(rear)
5. STOP
```

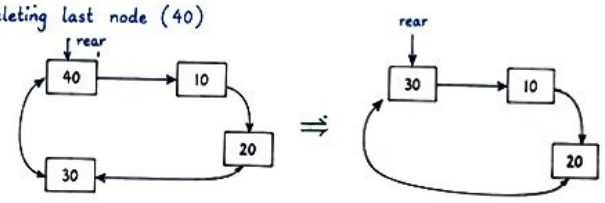
Explanation :

If only one node, make list empty.

Otherwise, find the second last node, make it point to the first node and delete the last node.

Diagrams :

(a) Deleting last node (40)



8. DESTROY (Make List Empty)

Pseudo code :

```
1. START
2. IF rear = NULL
       PRINT "List is already empty"
       STOP
3. SET temp = rear -> next
4. IF <= end :
   STOP
4. REPEAT
       SET del = temp
       FREE(del)
   UNTIL temp = rear -> next
5. rear = NULL
```

Explanation :

Delete all nodes one by one and finally set rear = NULL.

Diagram :

